Content

이번 주차의 학습목표와 학습내용을 확인하세요.

01. 스택의 개념과 동작 원리

스택의 개념과 동작 원리를 설명할 수 있다.

02. 스택의 응용

스택의 구현방식 차이를 설명하고 응용할 수 있다.

스택의
개념과 동작 원리

스택의 기본 개념

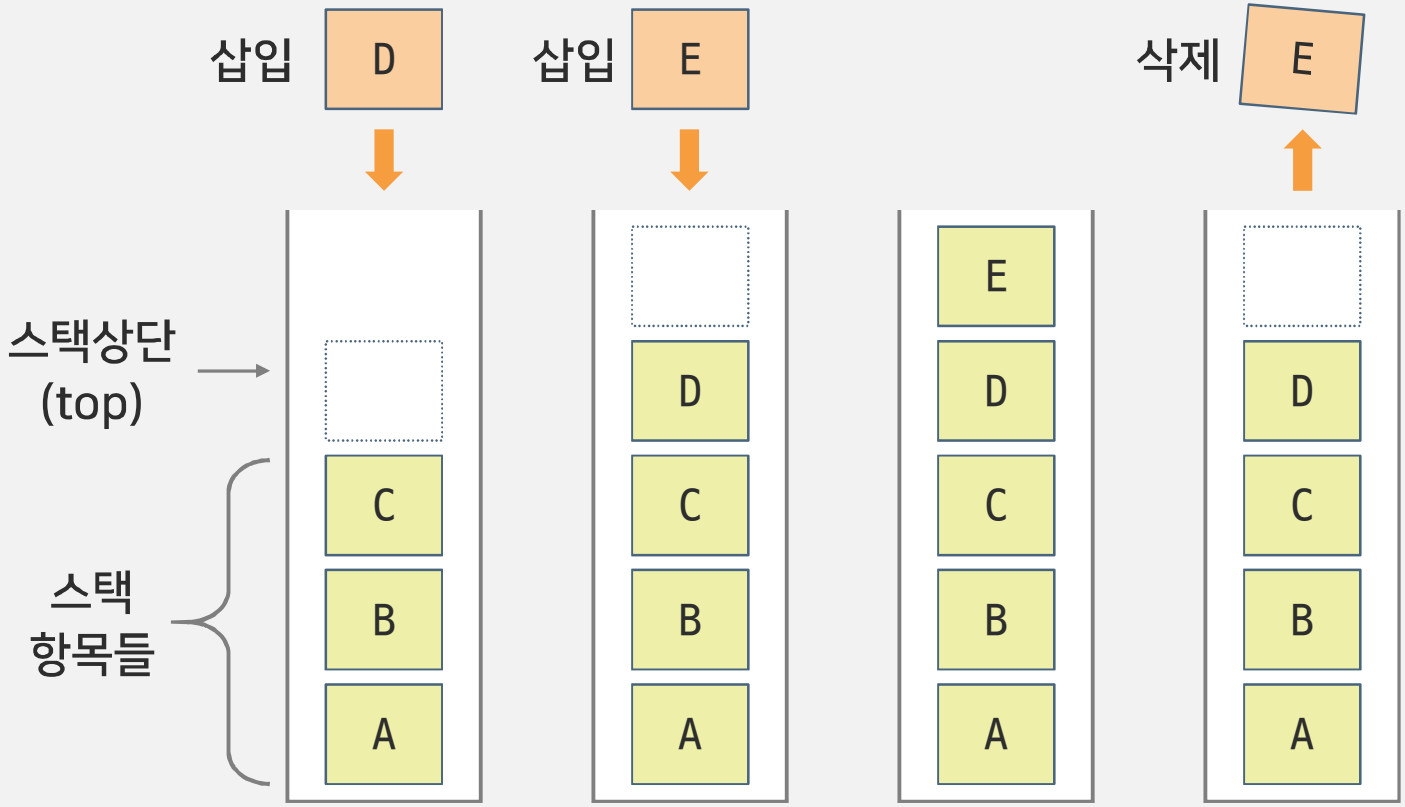
스택(stack)

스택	• 쌓아놓은 더미
후입선출 (LIFO, Last-In First-Out)	• 가장 최근에 들어온 데이터가 가장 먼저 나감

<이미지 출처: 최명규, 천인국. '파이썬으로 쉽게 풀어쓴 자료구조'. 생능출판사>

스택의 기본 개념

스택의 구조와 연산



스택의 기본 개념

스택 ADT

데이터

- 후입선출(LIFO)의 접근 방법을 유지하는 항목들의 모음

연산

- `Stack()`: 비어 있는 새로운 스택을 만듦
- `isEmpty()`: 스택이 비어있으면 True를 아니면 False를 반환함
- `push(e)`: 항목 e를 스택의 맨 위에 추가함
- `pop()`: 스택의 맨 위에 있는 항목을 꺼내 반환함
- `peek()`: 스택의 맨 위에 있는 항목을 삭제하지 않고 반환함
- `size()`: 스택 내의 모든 항목들의 개수를 반환함
- `clear()`: 스택을 공백상태로 만듦

스택의 기본 개념

스택의 용도

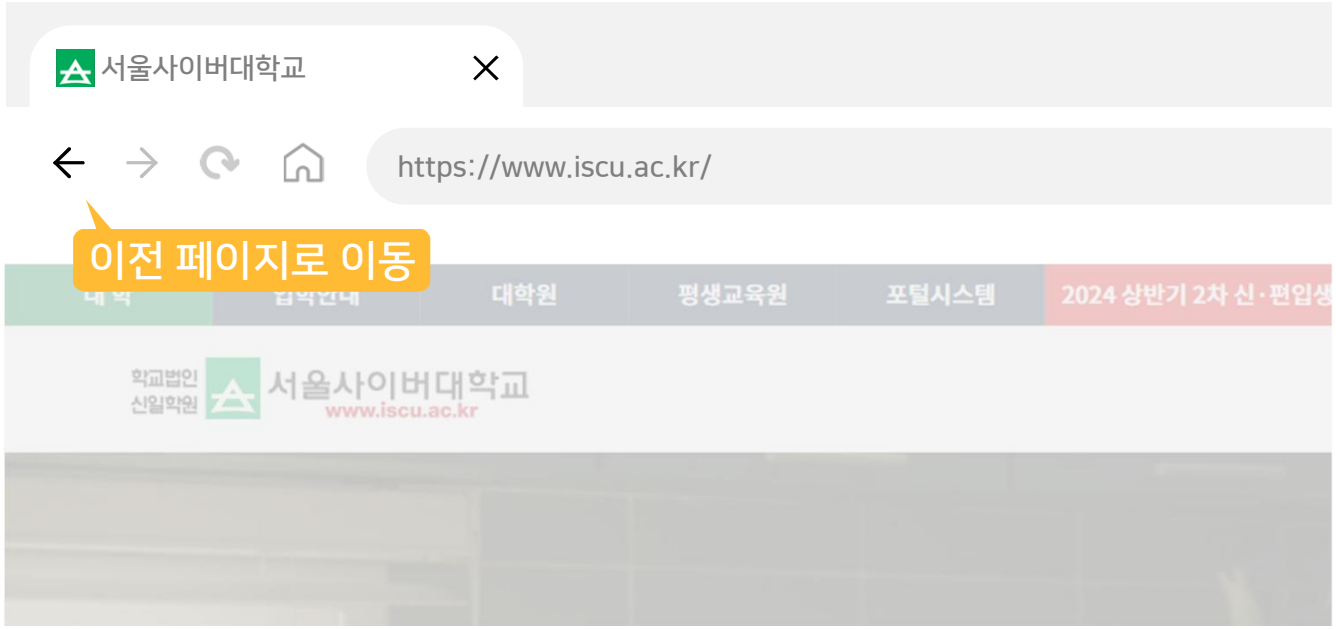
스택의 기본 개념

스택의 용도

함수 호출

미로탐색

되돌리기

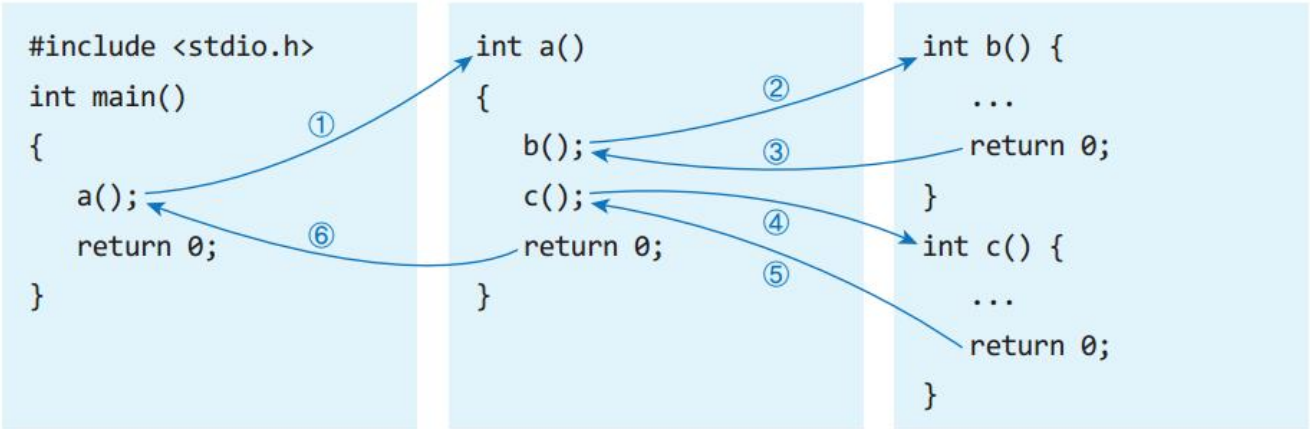


스택의 기본 개념

스택의 용도

함수 호출

미로탐색



스택의 구현

배열을 이용한 구현

top: 스택 항목을 저장하는 파이썬 리스트 (항목의 개수 = len(top))

isEmpty() : 공백상태이면 top이 0을 가리킴
isFull() : 포화상태이면 top은 capacity
push(e) : 포화상태(삽입가능 상태인지) 검사 필요 / 삽입 후 top한 칸 증가
pop() : 공백상태(삭제가능 상태인지) 검사 필요 / 삭제 후 top한 칸 감소
peek() : 공백상태 검사 필요 / 상단의 항목을 단순 반환

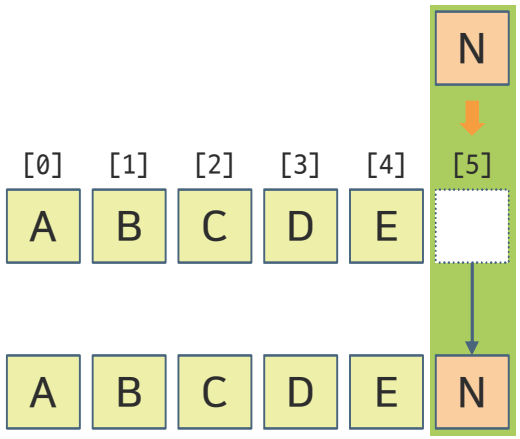
스택의 구현

배열을 이용한 구현

항목 삽입 · 삭제 위치	리스트의 맨 뒤가 유리

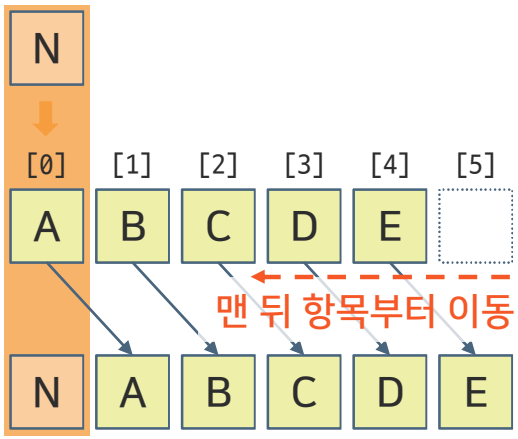
후단을 사용하는 경우

- 바로 추가하면 됨
- O(1)



전단을 사용하는 경우

- 많은 항목들의 이동이 필요(비효율적)
- O(n)



스택의 구현

함수를 이용한 구현

```
top = []                                  # 스택의 데이터: 항목을 위한 공백 리스트
def isEmpty():
    return len(top) == 0                  # len(top) == 0 의 계산 결과가 True/False
def push(item):
    top.append(item)                      # 리스트의 맨 뒤에 item을 추가함
def pop():
    if not isEmpty():                     # 공백상태가 아니면
       return top.pop(-1)                 # 리스트의 맨 뒤에서 항목을 하나 꺼내고 반환
def peek():                               # 맨 위의 항목을 삭제하지 않고 반환
    if not isEmpty():                     # 공백상태가 아니면
       return top[-1]                     # 맨 뒷 항목을 반환(삭제하지 않음)
def size(): return len(top)               # 스택의 크기
def clear():
    global top                            # top은 전역변수임을 지정함
    top = []                              # 스택의 초기화
```

스택의 구현

함수를 이용한 구현

활용

```
for i in range (1,6):                    # i = 1, 2, 3, 4, 5
    push(i)                              # push 연산 5회
print('push 5회: ', top)                  # 스택 내용 출력
print('pop() --> ', pop())               # pop연산 및 반환 항목 출력
print('pop() --> ', pop())               # pop연산 및 반환 항목 출력
print('pop  2회: ', top)                  # 스택 내용 출력 (str(top)도 동일함)
```

C:\WINDOWS\system32\cmd.exe

```
push 5회:  [1, 2, 3, 4, 5]
pop() -->  5
pop() -->  4
pop  2회:  [1, 2, 3]
```

가장 최근에 삽입된 항목

가장 먼저 삽입된 항목

스택의 구현

함수를 이용한 구현

활용

```
push('홍길동')                    # push 연산 2회
push('이순신')                    # push 연산 2회
print('push 2회: ', top)          # 스택 내용 출력
print('pop() --> ', pop())        # pop연산 및 반환 항목 출력
print('pop  1회: ', top)          # 스택 내용 출력 (str(top)도 동일함)
```

C:\WINDOWS\system32\cmd.ex

정수 문자열

```
push 2회:  [1, 2, 3, '홍길동', '이순신']
pop() -->  이순신
pop  1회:  [1, 2, 3, '홍길동']
```

스택의 구현

클래스를 이용한 구현

```
class Stack:
    def __init__(self):                    # 생성자
        self.top = []                      # top이 이제 클래스의 멤버 변수가 됨

    def isEmpty(self): return len(self.top) == 0
    def size(self): return len(self.top)
    def clear(self): self.top = []         # 주의: 이제 전역변수 선언이 필요없음


    def push(self, item ):
        self.top.append(item)

    def pop(self):
        if not self.isEmpty():
            return self.top.pop(-1)

    def peek(self):
        if not self.isEmpty():
            return self.top[-1]
```

스택의 구현

클래스를 이용한 구현

활용

```
odd = Stack()                                   # 홀수 저장을 위한 스택
even = Stack()                                  # 짝수 저장을 위한 스택
for i in range(10):                             # i = 0, 1, 2, …, 9
    if i%2 == 0 : even.push(i)                  # 짝수는 even에 push
    else : odd.push(i)                          # 홀수는 even에 push
print('스택 even push 5회: ', even)             # even 스택 출력
print('스택 odd  push 5회: ', odd)              # odd 스택 출력
print('스택 even     peek: ', even.peek())      # even 스택 peek()
print('스택 odd      peak: ', odd.peek())       # odd 스택 peek()
for_in range(2) : even.pop()                    # even 스택에서 두 번 pop()
for_in range(3) : odd.pop()                     # odd 스택에서 세 번 pop()
print('스택 even  pop 2회: ', even)             # even 스택 출력
print('스택 odd   pop 3회: ', odd)              # odd 스택 출력
```

스택의 구현

클래스를 이용한 구현

활용

스택 내용이 아니라
스택 객체의 정보가 출력됨

```
C:\WINDOWS\system32\cmd.exe
스택 even push 5회: <__main__.Stack object at 0x0000024A32CD3320>
스택 odd  push 5회: <__main__.Stack object at 0x0000024A32CD3208>
스택 even      peek: 8
스택 odd       peak: 9
스택 even  pop 2회: <__main__.Stack object at 0x0000024A32CD3320>
스택 odd   pop 3회: <__main__.Stack object at 0x0000024A32CD3208>
```

스택의 구현

스택의 출력

방법 1

```
print('스택 even push 5회: ', even.top)          # even 스택 출력
print('스택 odd  push 5회: ', odd.top)           # odd 스택 출력
```

C:\WINDOWS\system32\cmd.exe

```
스택 even push 5회: [0, 2, 4, 6, 8]
스택 odd  push 5회: [1, 3, 5, 7, 9]
스택 even      peek: 8
스택 odd       peak: 9
스택 even  pop 2회: [0, 2, 4]
스택 odd   pop 3회: [1, 3]
```

스택의 내용이 정상적으로 출력됨

스택의 구현

스택의 출력

방법 2 연산자 중복 및 슬라이싱

```
def __str__( self ):
    return str(self.top[::-1])        # 역순으로 출력. 최근의 항목을 먼저.
```

```
C:\WINDOWS\system32\cmd.exe
스택 even push 5회: [8, 6, 4, 2, 0]
스택 odd  push 5회: [9, 7, 5, 3, 1]
스택 even      peek: 8
스택 odd       peak: 9
스택 even  pop 2회: [4, 2, 0]
스택 odd   pop 3회: [3, 1]
```

내용이 역순으로 출력됨
최근에 삽입된 항목이 먼저 출력됨

스택의 응용

괄호검사

괄호의 종류

대괄호	중괄호	소괄호
[ ]	{ }	( )

조건 1 왼쪽 괄호의 개수와 오른쪽 괄호의 개수가 같아야 함

조건 2 같은 타입의 괄호에서 왼쪽 괄호가 오른쪽 괄호보다 먼저 나와야 함

조건 3 서로 다른 타입의 괄호 쌍이 서로 교차하면 안 됨

괄호검사

괄호의 사용

`{ A[(i+1)]=0; }`	➡	오류 없음
`if ((i==0) && (j==0)`	➡	오류: 조건 1 위반
`while (it < 10)) { it--; }`	➡	오류: 조건 2 위반
`A[(i+1])=0;`	➡	오류: 조건 3 위반

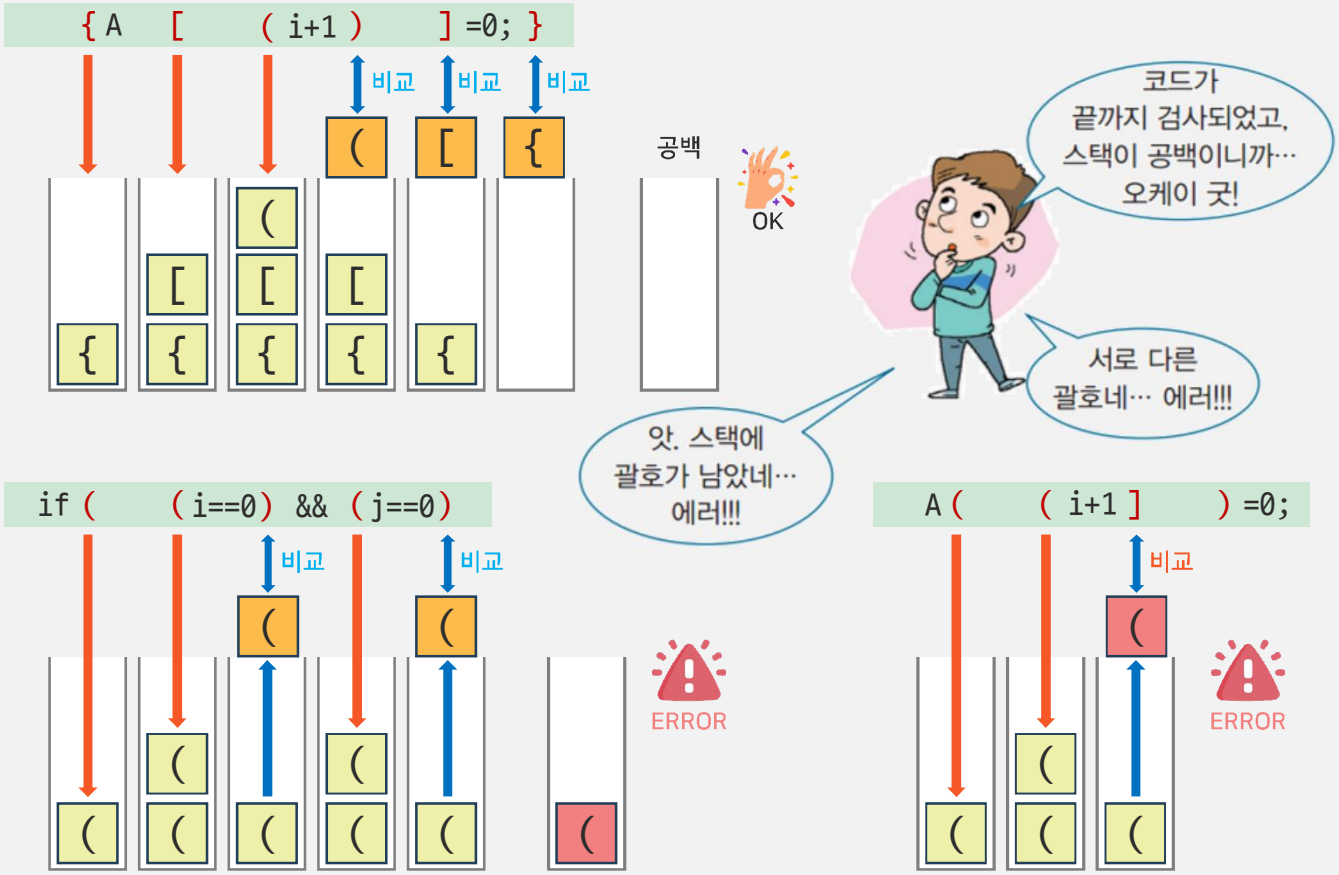
괄호검사

괄호검사 방법

1. 문자를 저장하는 스택을 준비함(처음에는 공백 상태가 되어야 함)
2. 입력 문자열의 문자를 하나씩 읽어 왼쪽 괄호를 만나면 스택에 삽입함
3. 오른쪽 괄호를 만나면 pop()연산으로 가장 최근에 삽입된 괄호를 꺼냄
   - 이 때 스택이 비었으면 조건 2에 위배됨
   - 꺼낸 괄호가 오른쪽 괄호와 짝이 맞지 않으면 조건 3에 위배됨
4. 끝까지 처리했는데 스택에 괄호가 남아 있으면 조건 1에 위배됨

괄호검사

사용 예시



<이미지 출처: 최명규, 천인국. '파이썬으로 쉽게 풀어쓴 자료구조'. 생능출판사>

괄호검사

알고리즘

```
def checkBrackets(statement):
    stack = Stack()
    for ch in statement:                                # 문자열의 각 문자에 대해
        if ch in ('{', '[', '('):                       # in '{[('도 동일하게 동작함
            stack.push(ch)
        elif ch in ('}', ']', ')'):                     # in '}])'도 동일하게 동작함
            if stack.isEmpty() :
                return False                            # 조건 2 위반
            else :
                left = stack.pop()
                if (ch == "}" and left != "{") or \
                    (ch == "]" and left != "[") or \
                    (ch == ")" and left != "(") :
                    return False                        # 조건 3 위반

    return stack.isEmpty()                              # False이면 조건 1위반
```

괄호검사

테스트 코드 1

```
str = ("{ A[(i+1)] = 0;}", "if((i==0) && (j==0)", "A[ (i+1] ) = 0;")
for s in str:
    m = checkBrackets(s)
    print(s," ---> ", m)
```

C:\WINDOWS\system32\cmd.exe

```
{ A[(i+1)] = 0; }  --->  True
if((i==0) && (j==0)  --->  False
A[ (i+1] ) = 0;  --->  False
```

괄호검사

테스트 코드 2

```
filename = "ArrayStack.h"                          # 괄호를 검사할 소스 파일
infile = open(filename, "r")                       # 파일 열기
lines = infile.readlines();                        # 파일 전체를 라인별로 읽은 리스트
infile.close()                                     # 파일 닫기

result = checkBracketsV2(lines)                    # 검사 함수 호출(약간 수정 필요)
print(filename, " ---> ", result)                  # 결과 출력
```

```
def checkBracketsV2(lines):
    stack = Stack()
        for line in lines:                         # 리스트의 모든 문자열에 대해
            for ch in line:                        # 문자열의 각 문자에 대해
                ...
```

C:\WINDOWS\system32\cmd.exe

```
ArrayStack.h  --->  True
계속하려면 아무 키나 누르십시오
```

계산기

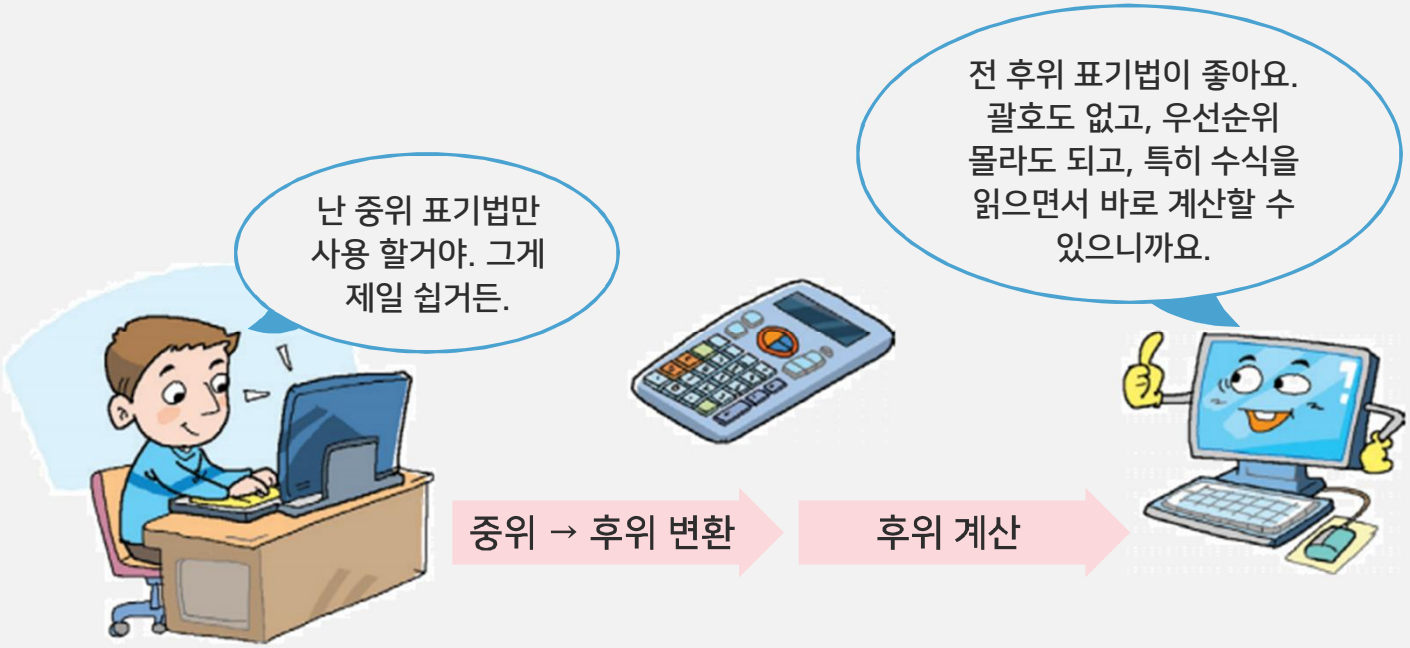
수식 표기 방법

전위(prefix)	중위(infix)	후위(postfix)
연산자 피연산자1 피연산자2	피연산자1 연산자 피연산자2	피연산자1 피연산자2 연산자
+ A B	A + B	A B +
+ 5 * A B	5 + A * B	5 A B * +

계산기

수식 계산 과정

계산기 프로그램을 만드는 과정

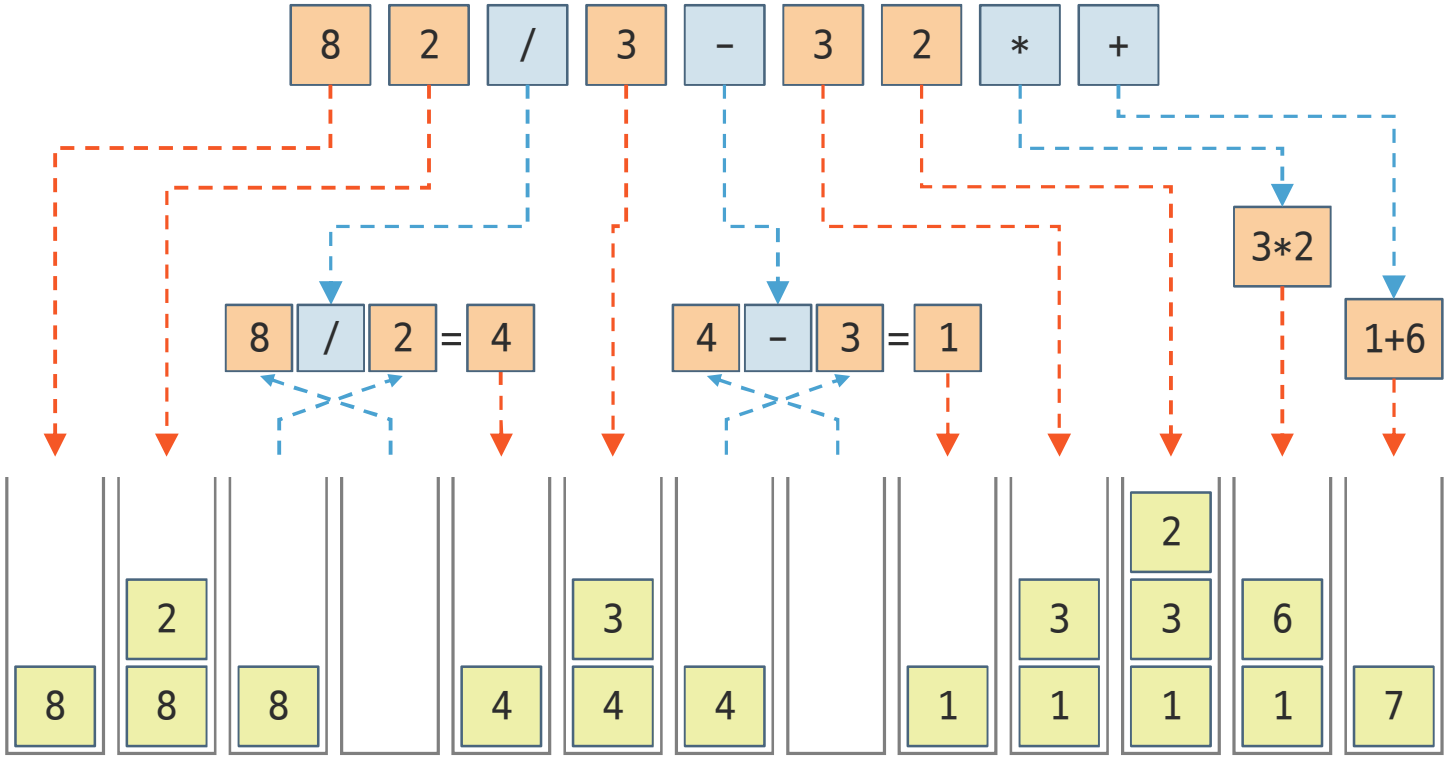


<이미지 출처: 최명규, 천인국. '파이썬으로 쉽게 풀어쓴 자료구조'. 생능출판사>

계산기

후위표기 수식의 계산

8 2 / 3 – 3 2 * +



계산기

알고리즘

```
def evalpostfix( expr ):
    s = Stack()                                  # 스택 객체 생성
    for token in expr :                          # 리스트의 모든 항목에 대해
        if token in “+-*/” :                     # 항목이 연산자이면
           val2 = s.pop()                                # 피연산자2
           val1 = s.pop()                                # 피연산자1
           if (token == ‘+’): s.push(val1 + val2)        # 각 연산 수행
           elif (token == ‘-’): s.push(val1 - val2)      # 결과는 스택에
           elif (token == ‘*’): s.push(val1 * val2)      # 다시 저장
           elif (token == ‘/’): s.push(val1 / val2)
        else :                                   # 항목이 피연산자이면
           s.push( float(token) )                # 실수로 변경해서 스택에 저장

    return s.pop()                               # 최종 결과를 반환
```

계산기

테스트 코드

```
expr1 = [ '8', '2', '/', '3', '-', '3', '2', '*', '+']
expr2 = [ '1', '2', '/', '4', '*', '1', '4', '/', '*']
print(expr1, '-->', evalPostfix(expr1))
print(expr2, '-->', evalPostfix(expr2))
```

C:\WINDOWS\system32\cmd.exe

```
['8', '2', '/', '3', '-', '3', '2', '*', '+'] --> 7.0
['1', '2', '/', '4', '*', '1', '4', '/', '*'] --> 0.5
```

계산기

중위 표기 수식의 후위 표기 변환

중위 표기와 후위 표기 비교

공통점

- 피연산자의 순서가 동일함

차이점

- 연산자들의 순서가 다름 (우선순위순서)
  - ➡ 연산자만 스택에 저장했다가 출력
  - ➡ 2+3*4 → 234*+

계산기

중위 표기 수식의 후위 표기 변환

변환 방법

1. 피연산자를 만나면 그대로 출력
2. 연산자를 만나면
   - 스택이 비어있으면 스택에 저장
   - 스택에 저장된 연산자의 우선순위가 낮으면 스택에 저장
   - 스택에 저장된 연산자의 우선순위가 높거나 같으면 스택에 저장된 연산자를 출력한 후 스택에 저장
3. 왼쪽 괄호를 만나면 스택에 저장
4. 오른쪽 괄호를 만나면 스택에서 왼쪽 괄호 위에 쌓여있는 모든 연산자를 출력

계산기

중위 표기 수식의 후위 표기 변환

A + B * C

단계	중위표기 수식	스택(우측이 상단)	후위표기 수식
0	A + B * C	[]	
1	A + B * C	[]	A
2	A + B * C	['+']	A
3	A + B * C	['+']	A B
4	A + B * C	['+', '*']	A B
5	A + B * C	['+']	A B C
6	A + B * C	[]	A B C * +

계산기

중위 표기 수식의 후위 표기 변환

A * B + C

단계	중위표기 수식	스택(우측이 상단)	후위표기 수식
0	A * B + C	[]	
1	A * B + C	[]	A
2	A * B + C	['*']	A
3	A * B + C	['*']	A B
4	A * B + C	['+']	A B *
5	A * B + C	['+']	A B * C
6	A * B + C	[]	A B * C +

계산기

중위 표기 수식의 후위 표기 변환

(A + B) * C

단계	중위표기 수식	스택(우측이 상단)	후위표기 수식
0	(A + B) * C	[]	
1	(A + B) * C	['(']	
2	(A + B) * C	['(']	A
3	(A + B) * C	['(', '+']	A
4	(A + B) * C	['(', '+']	A B
5	(A + B) * C	[]	A B +
6	(A + B) * C	['*']	A B +
7	(A + B) * C	['*']	A B + C
8	(A + B) * C	[]	A B + C *

계산기

변환 알고리즘

```
def precedence (op):
    if   op=='(' or op==')' : return 0
    elif op=='+' or op=='-' : return 1
    elif op=='*' or op=='/' : return 2
    else : return -1
```

계산기

변환 알고리즘

```
def Infix2Postfix( expr ):                  # expr: 입력 리스트(중위 표기식)
    s = Stack()
    output = []                             # output: 출력 리스트(후위 표기식)
    for term in expr :
        if term in '(' :                    # 왼쪽 괄호이면
           s.push('(')                      # 스택에 삽입
        elif term in ')' :                  # 오른쪽 괄호이면
           while not s.isEmpty():
              op = s.pop()
              if op=='(' : break;           # 왼쪽 괄호가 나올 때 까지
              else :                        # 스택에서 연산자를 꺼내 출력
                  output.append(op)
```

계산기

변환 알고리즘

```
    elif term in "+-*/" :                # 연산자이면
        while not s.isEmpty() :          # 우선순위가 높거나 같은 연산자를
            op=s.peek()                  # 스택에서 모두 꺼내 출력
            if( precedence(term) <= precedence(op)):
                output.append(op)
                s.pop()
            else: break
        s.push(term)                     # 마지막으로 현재 연산자 삽입
    else :                               # 피연산자이면
        output.append(term)              # 바로 출력


while not s.isEmpty() :                  # 처리가 끝났으면 스택에 남은 항목을
    output.append(s.pop())               # 모두 출력

return output                            # 결과(후위표기식 리스트)를 반환
```

계산기

테스트 코드

```
infix1 = ['8', '/', '2', '-', '3', '+', '(', '3', '*', '2', ')']
infix2 = ['1', '/', '2', '*', '4', '*', '(', '1', '/', '4', ')']
postfix1 = Infix2Postfix(infix1)
postfix2 = Infix2Postfix(infix2)
result1 = evalPostfix(postfix1)
result2 = evalPostfix(postfix2)
print(' 중위표기: ', infix1)
print(' 후위표기: ', postfix1)
print(' 계산결과: ', result1, end='\n\n')
print(' 중위표기: ', infix2)
print(' 후위표기: ', postfix2)
print(' 계산결과: ', result2)
```

계산기

테스트 코드

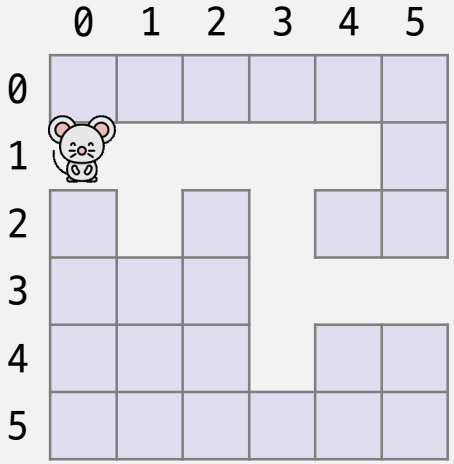
```
C:\WINDOWS\system32\cmd.exe

중위표기: ['8', '/', '2', '-', '3', '+', '(', '3', '*', '2', ')']
후위표기: ['8', '2', '/', '3', '-', '3', '2', '*', '+']
계산결과: 7.0

중위표기: ['1', '/', '2', '*', '4', '*', '(', '1', '/', '4', ')']
후위표기: ['1', '2', '/', '4', '*', '1', '4', '/', '*']
계산결과: 0.5
```

미로탐색

미로탐색의 의미

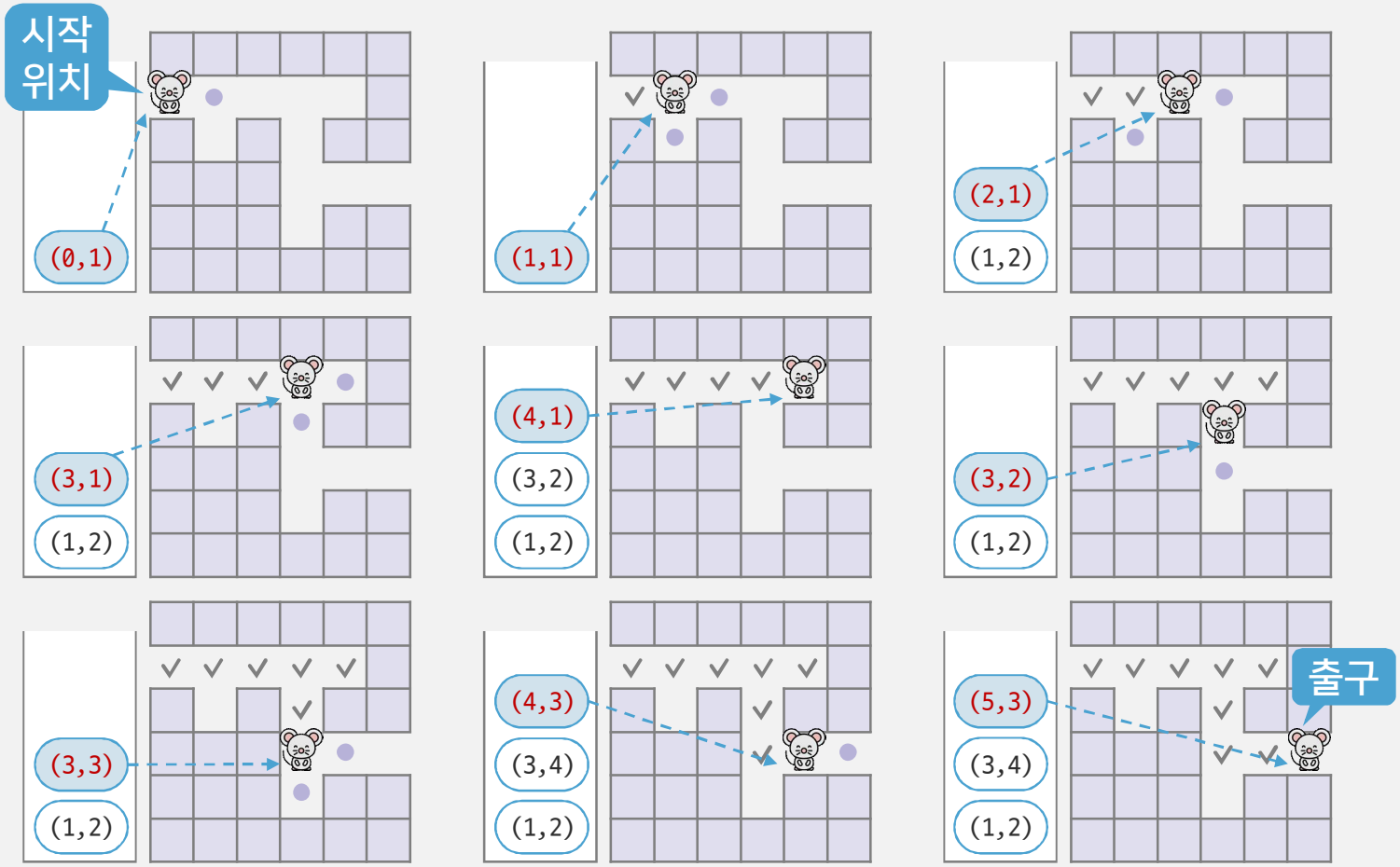



```
map = [ ['1','1','1','1','1','1'],
        ['e','0','0','0','0','1'],
        ['1','0','1','0','1','1'],
        ['1','1','1','0','0','X'],
        ['1','1','1','0','1','1'],
        ['1','1','1','1','1','1']]
MAZE_SIZE = 6
```

<이미지 출처: 최명규, 천인국. '파이썬으로 쉽게 풀어쓴 자료구조'. 생능출판사>

미로탐색

깊이우선탐색(Depth First Search)



미로탐색

알고리즘

```
def isValidPos(x, y):                    # (x, y) 가 갈 수 있는 곳인지 검사하는 함수
    if x < 0 or y < 0 or x >= MAZE_SIZE or y >= MAZE_SIZE :
        return False                     # (x, y) 가 미로 밖이면 갈 수 없음
    else :                               # 방('0') 또는 출구('X')면 갈 수 있음
        return map[y][x]='0' or map[y][x]='X'

def DFS():                               # 깊이우선탐색 함수
    stack = Stack()                      # 사용할 스택 객체를 준비
    stack.push( (0,1) )                  # 시작위치 삽입. (0,1)은 튜플
    print('DFS: ')
```

미로탐색

알고리즘

```
while not stack.isEmpty():        # 공백이 아닐 동안
    here = stack.pop()            # 항목을 꺼냄(pop)
    print(here, end='->')
    (x,y) = here                  # 스택에 저장된 튜플은 (x,y) 순서임
    if (map[y][x] == 'X'):        # 출구이면 탐색 성공. True 반환
        return True
    else :
        map[y][x] = '.'           # 현재 위치를 지나왔다고 '.'표시
        # 4방향의 이웃을 검사해 갈 수 있으면 스택에 삽입
        if isValidPos(x, y - 1): stack.push((x, y - 1)) # 상
        if isValidPos(x, y + 1): stack.push((x, y + 1)) # 하
        if isValidPos(x - 1, y): stack.push((x - 1, y)) # 좌
        if isValidPos(x + 1, y): stack.push((x + 1, y)) # 우
    print('현재 스택: ', stack)  # 현재 스택 내용 출력
return False                      # 탐색 실패. False 반환
```

미로탐색

테스트 코드

```
result = DFS()
if result : print(' --> 미로탐색 성공')
else : print(' --> 미로탐색 실패')
```

C:\WINDOWS\system32\cmd.exe

최종 탐색 순서

가장 최근에 삽입된 항목이 먼저 출력되도록 함

```
DFS
(0, 1)-> 현재 스택: [(1, 1)]
(1, 1)-> 현재 스택: [(2, 1), (1, 2)]
(2, 1)-> 현재 스택: [(3, 1), (1, 2)]
(3, 1)-> 현재 스택: [(4, 1), (3, 2), (1, 2)]
(4, 1)-> 현재 스택: [(3, 2), (1, 2)]
(3, 2)-> 현재 스택: [(3, 3), (1, 2)]
(3, 3)-> 현재 스택: [(4, 3), (3, 4), (1, 2)]
(4, 3)-> 현재 스택: [(5, 3), (3, 4), (1, 2)]
(5, 3)-> --> 미로탐색 성공
```

학습평가

학습한 내용을 바탕으로 다음 문제를 풀어보세요.

번호	문제	보기	
1	스택에서 사용되는 정보의 입출력 방법으로 옳은 것은?	① FIFO ② FILO ③ LIFO ④ LILO	
2	공백 상태 여부가 먼저 확인되어야 하는 스택의 연산은?	① push(), pop() ② push(), peek() ③ size(). clear() ④ pop(), peek()	
3	배열로 구현한 스택에서, top이 스택의 용량인 capacity와 같은 경우를 포화상태로 정의하였다. 이 스택의 공백 상태로 옳은 것은 ?	① top == 0 ② top == -1 ③ top < capacity ④ top == capacity -1	
4	스택의 연산에서 pop() 과 peek() 의 차이로 옳은 것은?	① pop()연산은 공백상태와 관계없이 연산을 수행 ② peek()연산은 맨뒤의 항목을 삭제하지 않고 반환 ③ peek()연산은 항목을 삭제하고 반환 ④ pop()연산은 맨 아래의 항목을 삭제하고 반환	

학습평가

학습한 내용을 바탕으로 다음 문제를 풀어보세요.

번호	문제	보기	
5	괄호검사 조건으로 옳지 않은 것은?	① 왼쪽 괄호의 개수와 오른쪽 괄호의 개수가 같아야 함 ② "[ ]", "{ }", "( )" 순으로 괄호의 우선순위가 존재함 ③ 같은 타입의 괄호에서 왼쪽 괄호가 오른쪽 괄호보다 먼저 나와야 함 ④ 서로 다른 타입의 괄호 쌍이 서로를 교차해서는 안됨	
6	다음 후위(postfix) 표기식의 연산 결과는?	① 35 ② 77 ③ 42 ④ 360	
7	다음 전위(prefix) 표기식의 연산 결과는?	① 29 ② -19 ③ 7 ④ 28	
8	중위 표기를 후위 표기로 변환할 때 스택에 저장되는 것으로 옳은 것은?	① 피연산자 ② 괄호를 제외한 연산자 ③ 닫히는 괄호와 연산자 ④ 열리는 괄호와 연산자	

학습정리

스택

스택의 개념과 동작 원리

- 스택은 무더기나 더미를 의미하며 선형 자료구조의 일종임
- 스택의 입출력은 후입선출의 형식으로 가장 최근에 들어온 데이터가 가장 먼저 나감
- 스택의 연산으로는 push(), pop(), peek(), isEmpty(), clear() 등이 있음
- push() 연산은 포화상태 검사가 필요하고, pop()과 peek()연산은 공백상태 검사가 필요함
- 클래스를 이용한 스택의 구현에서 capacit가 최대용량이고 top이 상단이라면, 포화상태일 때 top은 capacity 이고 공백상태일때 top은 0을 가리킴

스택의 응용

- 괄호검사의 조건
  ① 왼쪽 괄호의 개수와 오른쪽 괄호의 개수가 같아야 함
  ② 같은 타입의 괄호에서 왼쪽 괄호가 오른쪽 괄호보다 먼저 나와야 함
  ③ 서로 다른 타입의 괄호 쌍이 서로를 교차해서는 안 됨
- 수식의 표기법으로 전위법, 중위법, 후위법이 있음